

기사: 페이지 설정하기(파트 A)

article.php 페이지는 category.php와 유사한 제어 흐름을 사용한다. 만약에 쿼리 문자열이

- id라는 이름을 가지고 있다면, 그 값은 사용자가 편집하려는 기사의 아이디어야 한다.
- id라는 이름을 가지고 있지 않다면, 이 페이지는 새로운 기사를 생성하기 위해 사용되고 있음을 의미한다.

1. 엄격한 타입을 선언하고 필요한 파일을 인클루드한다.

2. 업로드 폴더 경로는 \$uploads(306쪽 참고)에 있다. 허용되는 이미지 타입은 \$file_types에 저장된다. 허용되는 파일 확장자는 \$file_exts에 저장된다. 최대 파일 크기는 \$max_size에 저장된다.

3. filter_input() 함수는 쿼리 문자열에 id라는 이름이 있는지 확인한다. 유효한 정수를 가지고 있다면 그 값은 \$id에 저장된다. 유효하지 않은 값을 가지고 있다면 \$id에는 false가 저장된다. 값이 존재하지 않다면 \$id에는 null이 저장된다.

4. 파일이 업로드되었는지 확인하기 위해, 이 페이지는 파일의 임시 위치를 가져와서 \$temp에 저장한다. 널 병합 연산자는 이미지가 업로드되지 않았을 경우 \$temp에 빈 문자열을 저장하는 데 사용된다.

5. \$destination 변수가 초기화된다. 이미지가 업로드되었다면 이미지가 저장되어야 하는 경로를 이 변수가 가지도록 업데이트될 것이다.

6. \$article 배열이 선언되고 디폴트값으로 초기화된다. 이 디폴트값은 사용자가 새로운 기사를 생성하고자 할 때(하지만 사용자가 아직 값을 입력하지 않았을 때) 파트 D의 폼에 표시할 무언가를 제공하는 것이다. 다음 페이지의 예제 코드는 한 줄에 두 개의 요소를 선언하고 있다. 하지만 다운로드한 샘플 코드에서는 한 줄에 하나의 요소를 선언한다.

7. \$errors 배열은 빈 문자열로 초기화된다. 이 배열의 값은 각 폼 컨트롤 다음에 표시된다.

8. if 문은 쿼리 문자열에 아이디가 있는지 확인한다.

아이디가 있다면 페이지는 기사를 편집하며 기사 데이터를 데이터베이스에서 수집해야 한다.

9. \$sql은 기사 데이터를 가져오는 SQL 쿼리를 저장한다.

10. pdo() 함수는 SQL 문을 실행하며 PDOStatement 객체의 fetch() 메서드는 기사 데이터를 수집한다. 반환된 값은 6번에서 생성한 \$article 배열에 저장된 데이터를 덮어쓴다. 기사를 찾지 못한다면, \$article은 false를 갖게 될 것이다.

11. 만약에 \$article 변수가 false 값을 갖게 된다면, 사용자는 기사를 찾을 수 없다는 실패 메시지와 함께 articles.php로 리디렉션된다.

12. 기사에 대한 이미지가 저장되었다면, \$article 배열의 image_file 요소는 값을 가지므로 \$saved_image에는 true 값이 주어진다. 만약에 기사에 대한 이미지가 없다면 false 값이 주어진다.

13. 모든 작성자와 카테고리는 데이터베이스에서 수집된다. 이 값은 작성자와 카테고리를 나타내는 드롭 다운 박스를 생성하는 데 사용된다. 먼저 \$sql은 모든 회원의 아이디, 이름, 성을 가져오는 SQL 쿼리를 갖는다.

14. pdo() 함수는 쿼리를 실행하며 PDOStatement 객체의 fetchAll() 메서드는 결과를 수집하여 \$authors 변수에 저장한다. 인덱스 배열이 반환되며 각 요소의 값은 회원 상세 정보의 연관 배열이다.

15. 다음으로 \$sql 변수는 모든 카테고리의 아이디와 이름을 가져오는 쿼리를 갖는다.

16. pdo() 함수는 쿼리를 실행하며 PDOStatement 객체의 fetchAll() 메서드는 카테고리 데이터를 가져와서 \$categories에 저장한다.

```

<?php
// 파트 A: 설정하기
declare(strict_types = 1); // 엄격한 타입 사용
include '../includes/database-connection.php'; // 데이터베이스 연결
① include '../includes/functions.php'; // 함수
include '../includes/validate.php'; // 유효성 검사 함수
$uploads = dirname(__DIR__, 1) . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR;
② $file_types = ['image/jpeg', 'image/png', 'image/gif',]; // 허용되는 타입
$file_exts = ['jpg', 'jpeg', 'png', 'gif',]; // 허용되는 확장자
$max_size = 5242880; // 최대 파일 크기
// PHP 코드에 필요한 변수 초기화
③ $id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT); // 아이디 가져오기 + 유효성 검사
④ $temp = $_FILES['image']['tmp_name'] ?? ''; // 임시 이미지
⑤ $destination = ''; // 파일을 저장할 위치
// HTML 페이지에 필요한 변수 초기화
$article = [
    'id' => $id, 'title' => '',
    'summary' => '', 'content' => '',
⑥ 'member_id' => 0, 'category_id' => 0,
    'image_id' => null, 'published' => false,
    'image_file' => '', 'image_alt' => '',
]; // 기사 데이터
$errors = [
    'warning' => '', 'title' => '', 'summary' => '', 'content' => '',
⑦ 'author' => '', 'category' => '', 'image_file' => '', 'image_alt' => '',
]; // 오류 메시지
// 아이디가 있다면, 페이지는 기사 편집 중이므로 현재 기사 데이터 가져오기
⑧ if ($id) { // 아이디가 있다면
    $sql = "SELECT a.id, a.title, a.summary, a.content,
                a.category_id, a.member_id, a.image_id, a.published,
⑨                i.file AS image_file,
                i.alt AS image_alt
                FROM article AS a
                LEFT JOIN image AS i ON a.image_id = i.id
                WHERE a.id = :id;"; // 기사를 가져오는 SQL
    $article = pdo($pdo, $sql, [$id])->fetch(); // 기사 데이터 가져오기
    if (!$article) { // 기사가 없다면
⑪        redirect('articles.php', ['failure' => 'Article not found']); // 리디렉션
    }
}
⑫ $saved_image = $article['image_file'] ? true : false; // 이미지가 업로드되었는가?
// 모든 회원과 카테고리 가져오기
⑬ $sql = "SELECT id, forename, surname FROM member;"; // 모든 회원 가져오는 SQL
⑭ $authors = pdo($pdo, $sql)->fetchAll(); // 모든 회원 가져오기
⑮ $sql = "SELECT id, name FROM category;"; // 모든 카테고리 가져오는 SQL
⑯ $categories = pdo($pdo, $sql)->fetchAll(); // 모든 카테고리 가져오기

```


기사: 데이터 가져오고 유효성 검사하기(파트 B)

지금부터 두 페이지에서는 사용자가 보낸 데이터를 수집하고 유효성을 검증하는 파트 B에 대한 코드를 보여준다.

1. if 문은 폼이 게시되었는지 확인한다.
2. image_file 요소가 \$errors 배열에 추가되고 삼항 연산자가 값을 할당한다. 만약에 업로드하려는 파일의 크기가 php.ini 또는 .htaccess의 최대 업로드 파일 크기보다 커서 파일을 업로드할 수 없다면, 오류가 저장된다. 그렇지 않다면 빈 문자열이 저장된다.
3. if 문은 파일이 업로드되었고 오류가 없는지 확인한다. 만약에 그렇다면, 파일의 유효성을 검사할 것이다.
4. 이미지가 업로드되었기 때문에 대체 alt 텍스트가 수집되어 \$article 배열에 저장된다.
5. 이미지의 미디어 타입이 허용되는 파일 타입(이전 페이지의 2번에서 설정함)이면, 빈 문자열은 \$errors 배열의 image_file 요소의 값에 추가된다. 만약에 그렇지 않다면 오류 메시지가 추가된다.
6. 파일의 확장자가 허용(이전 페이지의 2번에서 설정함)되면 \$errors 배열의 image_file 요소의 값에 빈 문자열이 추가된다. 만약에 그렇지 않다면 오류 메시지가 추가된다.
7. 파일이 최대 크기(이전 페이지의 2번에서 설정함)보다 더 크다면, 파일 크기가 너무 크다는 메시지가 image_file의 값으로 추가된다.
8. image_alt라는 키가 \$errors 배열에 추가된다. 만약에 대체 텍스트의 길이가 1~254문자면 빈 문자열이 저장되며, 그렇지 않다면 오류 메시지가 저장된다.
9. if 문은 \$errors 배열의 image_file과 image_alt 키가 모두 비어 있는지 확인한다. 만약에 그렇다면 이미지가 처리될 수 있다.

10. create_filename() 함수(functions.php에 있는 함수이며 296~297쪽에서 설명함)는 파일 이름에서 원하지 않는 문자를 제거하고 고유한 이름인지 확인한다. 이 이름은 \$article 배열의 image_file 키에 저장된다.

11. \$desination 변수는 이미지가 업로드될 경로를 갖는다. 이 값은 업로드 폴더의 경로(이전 페이지의 2번에서 \$uploads 변수에 저장됨)와 이전 과정에서 생성된 파일 이름이 결합하여 생성된다.

12. 기사 데이터는 폼에서 수집된다. 기사가 업데이트되면 업데이트된 값은 이전 페이지의 9~10번에서 데이터베이스로부터 수집된 기존의 데이터를 덮어쓴다.

13. 기사를 게시하는 옵션은 체크박스다. 이것이 체크된 경우에만 서버로 전송한다. 체크되었다면 1의 값이 할당되고 체크되지 않았다면 0이 할당된다. 0과 1은 false와 true의 불리언값을 나타내기 위해 데이터베이스가 저장하는 값이기 때문이다.

14. 각 텍스트는 6장에서 생성한 함수로 유효성을 검증한다. 데이터가 유효하다면 해당 요소는 빈 문자열로 저장한다. 유효하지 않다면 해당 폼 컨트롤에 대한 오류 메시지를 저장한다.

15. is_member_id()와 is_category_id() 함수는 validate.php 파일에 추가된다. 제공된 값이 유효한지 확인하기 위해 이전 페이지의 13~16번에서 수집된 회원과 카테고리 배열을 가지고 루프를 돈다.

16. \$errors 배열의 값은 PHP의 implode() 함수를 사용하여 하나의 문자열로 결합하고 \$invalid라는 변수에 저장된다. 이것은 데이터를 데이터베이스에 저장해야 할지를 알려주는 데 사용된다.

// 파트 B: 폼 데이터 가져오고 유효성 검사하기

```

① if ($_SERVER['REQUEST_METHOD'] == 'POST') { // 폼이 제출되었다면
    // 파일이 php.ini 또는 .htaccess에서 제한된 크기보다 더 클 때의 오류 메시지
② $errors['image_file'] = ($_FILES['image']['error'] === 1) ? 'File too big ' : '';

    // 이미지가 업로드되었다면 이미지 데이터를 가져와서 유효성 검사함
③ if ($temp and $_FILES['image']['error'] === 0) { // 파일이 업로드되었다면
④     $article['image_alt'] = $_POST['image_alt']; // 대체 텍스트 가져오기
    // 이미지 파일 유효성 검사
    $errors['image_file'] .= in_array(mime_content_type($temp), $file_types)
        ? '' : 'Wrong file type. '; // 파일 타입 검사
    $ext = strtolower(pathinfo($_FILES['image']['name'], PATHINFO_EXTENSION));
    $errors['image_file'] .= in_array($ext, $file_extensions)
        ? '' : 'Wrong file extension. '; // 확장자 검사
    $errors['image_file'] .= ($_FILES['image']['size'] <= $max_size)
        ? '' : 'File too big. '; // 크기 검사
    $errors['image_alt'] = (is_text($article['image_alt'], 1, 254))
        ? '' : 'Alt text must be 1-254 characters.'; // 대체 텍스트 검사
    // 이미지 파일이 유효하다면 저장할 위치 지정
⑨ if ($errors['image_file'] === '' and $errors['image_alt'] === '') { // 유효하다면
⑩     $article['image_file'] = create_filename($_FILES['image']['name'], $uploads);
⑪     $destination = $uploads . $article['image_file']; // 목적지
    }

    // 기사 데이터 가져오기
    $article['title'] = $_POST['title']; // 제목
    $article['summary'] = $_POST['summary']; // 요약
    $article['content'] = $_POST['content']; // 내용
    $article['member_id'] = $_POST['member_id']; // 저자
    $article['category_id'] = $_POST['category_id']; // 카테고리
    $article['published'] = (isset($_POST['published'])
⑬ and ($_POST['published'] == 1)) ? 1 : 0; // 게시되었는가?

    // 기사 데이터의 유효성 검사하고 유효하지 않다면 오류 메시지 생성
    $errors['title'] = is_text($article['title'], 1, 80)
        ? '' : 'Title must be 1-80 characters';
    $errors['summary'] = is_text($article['summary'], 1, 254)
        ? '' : 'Summary must be 1-254 characters';
    $errors['content'] = is_text($article['content'], 1, 100000)
        ? '' : 'Article must be 1-100,000 characters';
    $errors['member'] = is_member_id($article['member_id'], $authors)
        ? '' : 'Please select an author';
    $errors['category'] = is_category_id($article['category_id'], $categories)
        ? '' : 'Please select a category';
⑮ $invalid = implode($errors);
⑯ // 오류 결합하기

```


기사: 변경사항 저장하기(파트 C)

지금부터 두 페이지에서는 파트 C에 대한 코드를 보여준다.

1. if 문은 \$invalid에 오류 메시지가 포함되어 있는지 확인한다. 만약에 포함하고 있다면, \$errors 배열은 사용자에게 폼 오류를 수정하라는 메시지를 저장한다.
2. 포함되어 있지 않다면, 데이터는 유효하며 처리할 수 있다.
3. \$article의 데이터는 \$arguments에 복사된다. pdo() 함수는 \$arguments의 값을 사용한다. 파트 D의 HTML 폼은 \$article의 값을 사용한다.
4. try 블록에는 데이터베이스를 업데이트하는 코드가 있다.
5. 기사를 생성하거나 업데이트하기 위해서는 두 개의 SQL 문이 필요하므로 트랜잭션이 시작된다. 하나의 SQL 문이 실패한다는 것은 두 개 모두가 실패한다는 것이다.
6. \$destination이 값을 가지고 있다면(513쪽에서 11번) 이미지는 업로드된 것이다. article 테이블은 이미지에 대해 생성된 아이디를 저장해야 하므로 기사 데이터보다 먼저 이미지가 처리되어야 한다.
7. Imagick는 이미지 크기를 조정하고 저장하는 데 사용된다.
 - Imagick 객체는 업로드된 이미지를 나타내기 위해 생성된다(해당 경로는 \$temp에 있음 - 511쪽에서 4번).
 - 1200 x 700픽셀로 크기가 조정된다.
 - 이미지는 \$destination의 경로에 저장된다.
8. \$sql은 데이터베이스의 image 테이블에 이미지의 파일명과 대체 텍스트를 추가하는 SQL 문을 갖는다.
9. pdo() 함수는 SQL 문을 실행한다. 인수(이미지 파일과 대체 텍스트)는 pdo() 함수에 인덱스 배열로 전달된다.
10. 이미지의 아이디는 PDO 객체의 lastInsertId() 메서드를 사용하여 수집되며 3번에서 생성한 \$arguments 배열의 image_id 키에 저장된다.

11. \$arguments 배열에서 image_file 키와 image_alt 키의 요소들은 제거된다. 왜냐하면 두 번째 SQL 문의 각 플레이스홀더에 대한 요소만 배열에 포함되어야 하기 때문이다.
12. if 문은 아이디가 지정되었는지 확인한다. 만약에 지정되어 있다면, 기사는 업데이트되는 중이며 \$sql 변수는 데이터베이스를 업데이트하는 SQL 문을 갖는다.
13. 만약에 아이디가 없다면, 새로운 기사가 생성되는 중이므로 \$arguments 배열에서 id를 제거하고, \$sql 변수는 새로운 기사를 데이터베이스에 추가하는 SQL을 갖는다.
14. pdo() 함수는 SQL 문을 실행한다.
15. PDO 객체의 commit() 메서드는 트랜잭션의 두 변경사항을 모두 데이터베이스에 저장하기 위해 호출된다.
16. 코드가 실행 중이면, 기사가 저장되었으니 redirect()는 사용자를 articles.php로 보낸다.
17. 만약에 try 블록에서 PDO가 예외를 발생하면 catch 블록이 실행된다. 예외 객체는 \$e에 저장된다.
18. PDO 객체의 rollBack() 메서드는 트랜잭션의 변경사항이 데이터베이스에 저장되는 것을 막는다.
19. 만약에 7번에서 이미지가 서버에 저장되었다면, PHP의 unlink() 메서드는 그 이미지를 삭제하기 위해 사용된다.
20. 만약에 PDOException 객체의 오류 코드가 1062면, 제목이 이미 사용 중이므로 \$errors에 오류가 저장된다.
21. 1062가 아니라면 예외가 다시 발생한다.
22. 이 줄의 코드가 실행된다면, 기사는 유효하지 않은 데이터를 가지고 있을 것이다. 기사에 이미지가 이미 있다면(511쪽에서 12번 참고), \$article 배열에 저장되어 있다. 만약에 기사에 이미지가 없었다면 image_file은 빈 문자열로 설정된다.

// 파트 C: 데이터가 유효한지 확인하고, 유효하다면 데이터베이스 업데이트하기

```

1  if ($invalid) { // 유효하지 않다면
2      $errors['warning'] = 'Please correct the errors below'; // 메시지 저장
3  } else { // 그렇지 않다면
4      $arguments = $article; // 기사 데이터 저장
5      try { // 데이터 삽입 시도
6          $pdo->beginTransaction(); // 트랜잭션 시작
7          if ($destination) { // 유효한 이미지가 있다면
8              $imagick = new \Imagick($temp); // Imagick 객체 생성
9              $imagick->cropThumbnailImage(1200, 700); // 자른 이미지 생성
10             $imagick->writeImage($destination); // 파일 저장
11             $sql = "INSERT INTO image (file, alt)
12                 VALUES (:file, :alt);"; // 이미지를 추가하는 SQL
13             pdo($pdo, $sql, [$arguments['image_file'], $arguments['image_alt']],);
14             $arguments['image_id'] = $pdo->lastInsertId(); // 새로운 이미지 아이디 얻기
15         }
16         unset($arguments['image_file'], $arguments['image_alt']); // 이미지 데이터 잘라내기
17         if ($id) {
18             $sql = "UPDATE article
19                 SET title = :title, summary = :summary, content = :content,
20                     category_id = :category_id, member_id = :member_id,
21                     image_id = :image_id, published = :published
22                 WHERE id = :id;"; // 기사 업데이트하는 SQL
23         } else {
24             unset($arguments['id']); // 아이디 제거
25             $sql = "INSERT INTO article (title, summary, content, category_id,
26                 member_id, image_id, published)
27                 VALUES (:title, :summary, :content, :category_id, :member_id,
28                     :image_id, :published);"; // 기사를 생성하는 SQL
29         }
30         pdo($pdo, $sql, $arguments); // 기사를 추가하는 SQL 실행
31         $pdo->commit(); // 변경사항 커밋
32         redirect('articles.php', ['success' => 'Article saved']); // 리디렉션
33     } catch (PDOException $e) { // PDOException이 발생했다면
34         $pdo->rollBack(); // SQL 변경사항 롤백
35         if (file_exists($destination)) { // 이미지 파일이 존재한다면
36             unlink($destination); // 이미지 파일 삭제
37         } // 예외가 PDOException이고 무결성 제약에 걸렸다면
38         if ($e->errorInfo[1] === 1062) {
39             $errors['warning'] = 'Article title already used'; // 경고를 저장
40         } else { // 그렇지 않다면
41             throw $e; // 예외를 다시 발생
42         }
43     }
44 }
45 // 새로운 이미지가 업로드되었지만 데이터가 유효하지 않다면, $article에서 이미지 제거
46 $article['image_file'] = $saved_image ? $article['image_file'] : ''; ...

```


기사: 폼/메시지(파트 D)

기사를 생성하거나 편집할 때 동일한 폼이 사용자에게 표시된다.

노트: 다운로드한 샘플 코드에서의 폼은 더 많은 HTML 요소와 폼 컨트롤에 레이블을 지정하고 폼의 표현을 제어하는 데 사용하는 속성을 가지고 있다. 다음 페이지는 중요한 코드가 한 페이지에 들어갈 수 있도록 하고, 수행 중인 작업에 집중할 수 있게 하기 위해 이러한 것이 제거되었다.

1. 여는 `<form>` 태그의 `action` 속성은 `article.php` 페이지를 가리킨다. 쿼리 문자열은 `id`라는 이름을 가지고 있다. 만약에 페이지가 기존의 기사를 업데이트한다면 해당 값은 기사의 아이디다(파일 상단에 있는 `$id` 속성에 저장됨). 데이터는 HTTP POST를 사용하여 전송된다.

2. `$errors` 배열의 `warning` 키에 대한 값이 있다면, 사용자에게 표시될 것이다.

3. 이미지를 업로드하기 위한 폼은 해당 기사에 아직 이미지가 제공되지 않은 경우에만 표시된다.

4. 파일 입력은 방문자가 이미지를 업로드할 수 있게 한다.

5. `$errors` 배열이 폼 컨트롤에 대한 오류 메시지를 가지고 있다면, 파일 입력 다음에 표시된다.

게시된 체크박스를 제외한 모든 폼 컨트롤 뒤에는 동일한 작업이 수행된다.

6. 텍스트 입력은 방문자가 대체 텍스트를 제공한다.

7. 이미지가 제공되었고 데이터가 유효하다면, 이미지는 페이지에 표시되며 그다음으로 대체 텍스트가 표시된다.

8. 그 아래에는 두 개의 링크가 있다. 첫 번째 링크는 사용자가 대체 텍스트를 편집할 수 있게 하며, 두 번째 링크는 이미지를 삭제한다.

9. 기사 제목은 텍스트 입력을 통해 제공된다. 값이 이미 제공되었다면 `value` 속성에 추가된다. 모든 예약 문자는 XSS 공격을 방지하기 위해 엔티티로 대체된다.

10. 요약은 `<textarea>` 요소를 사용한다. 만약에 요약이 제공되어 있다면, 예약된 문자를 엔티티로 대체하기 위해 `html_escape()` 함수를 사용하여 `<textarea>` 태그 사이에 작성한다.

11. 메인 기사 콘텐츠는 `<textarea>` 요소를 사용한다. 만약에 값이 제공되어 있다면, `<textarea>` 태그 사이에 작성한다.

12. 작성자 선택 박스에는 기사를 작성할 수 있는 모든 회원의 목록이 표시된다.

13. 이것은 모든 회원의 배열(파트 A에서 수집하여 `$authors`라는 변수에 저장함)을 가지고 `foreach` 루프를 사용하여 구축된다.

14. 각 회원에 `<option>` 요소가 추가된다.

15. 삼항 연산자의 조건은 작성자가 제공되었는지 그리고 그 작성자의 아이디가 선택 박스에 추가된 현재 작성자의 아이디와 일치하는지를 확인한다. 만약에 그렇다면, 기사의 작성자로 현재 회원이 선택되도록 `selected` 속성이 추가된다.

16. 회원의 이름은 옵션에 표시된다.

17. `$categories`에 저장된 카테고리의 배열은 카테고리에 대한 선택 박스를 생성하는 데 사용된다.

18. 체크박스는 기사가 게시될지(사이트에 표시될지)를 나타낸다. 삼항 연산자는 옵션이 선택되었는지를 검사한다. 만약에 그렇다면, `checked` 속성이 요소에 추가된다.

<!-- 파트 D - 폼 표시하기 -->

```

① <form action="article.php?id=<?=$id ?>" method="post" enctype="multipart/form-data">
    <h2>Edit Articles</h2>
    ② <?php if ($errors['warning']) { ?>
        <div class="alert alert-danger"><?=$errors['warning'] ?></div>
    <?php } ?>

    ③ <?php if (!$article['image_file']) { ?>
    ④     Upload image: <input type="file" name="image" class="form-control-file" id="image">
    ⑤     <span class="errors"><?=$errors['image_file'] ?></span>
    ⑥     Alt text: <input type="text" name="image_alt">
        <span class="errors"><?=$errors['image_alt'] ?></span>
    <?php } else { ?>
    ⑦     <label>Image:</label> "
        <p class="alt"><strong>Alt text:</strong> <?=$html_escape($article['image_alt']) ?></p>
    ⑧     <a href="alt-text-edit.php?id=<?=$article['id'] ?>">Edit alt text</a>
        <a href="image-delete.php?id=<?=$id ?>">Delete image</a><br><br>
    <?php } ?>

    ⑨ Title: <input type="text" name="title" value="<?=$html_escape($article['title']) ?>">
        <span class="errors"><?=$errors['title'] ?></span>
    ⑩ Summary: <textarea name="summary"><?=$html_escape($article['summary']) ?></textarea>
        <span class="errors"><?=$errors['summary'] ?></span>
    ⑪ Content: <textarea name="content"><?=$html_escape($article['content']) ?></textarea>
        <span class="errors"><?=$errors['content'] ?></span>
    ⑫ Author: <select name="member_id">
    ⑬     <?php foreach ($authors as $author) { ?>
    ⑭         <option value="<?=$author['id'] ?>"
    ⑮             <?=( $article['author_id'] == $author['id'] ) ? 'selected' : ''; ?>>
    ⑯             <?=$html_escape($author['forename'] . ' ' . $author['surname']) ?>
                </option>
    <?php } ?></select>
        <span class="errors"><?=$errors['author'] ?></span>
    ⑰ Category: <select name="category_id">
        <?php foreach ($categories as $category) { ?>
            <option value="<?=$category['id'] ?>"
                <?=( $article['category_id'] == $category['id'] ) ? 'selected' : ''; ?>>
                <?=$html_escape($category['name']) ?>
            </option>
        <?php } ?></select>
        <span class="errors"><?=$errors['category'] ?></span>
    ⑱ <input type="checkbox" name="published" value="1"
        <?=( $article['published'] == 1 ) ? 'checked' : '' ?>> Published
        <input type="submit" name="create" value="save" class="btn btn-primary">
</form>

```